UMLS
UNIFIED MODELING LANGUAGE SYSTEM

SEMESTER PROJECT

UNIVERSITY COURSE REGISTRATION SYSTEM

STREAMLINING ACADEMICS. EMPOWERING STUDENTS.
CONNECTING UNIVERSITY.

- STUDENT MANAGEMENT
- COURSE MANAGEMENT
- REGISTRATION MANAGEMENT
- REPORTS & ANALYTICS
- SECURE & RELIABLE

STUDENT 01

Laiba Amjad

Roll Number
FA24B1-SE-021

STUDENT 02

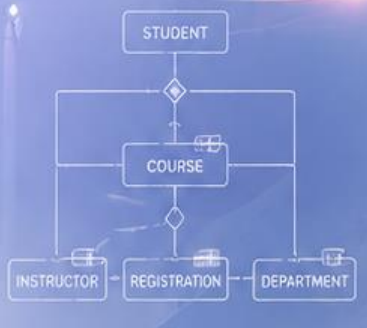
Maleeha Nadeem

Roll Number
FA26B1-SE-013

INSTRUCTOR

Sir Shahzad

UMLS
UNIFIED MODELING LANGUAGE SYSTEM



CONNECT | LEARN | SUCCEED

Project Scenario

This document presents the comprehensive object-oriented analysis and design for 'UniReg', a modern web-based Course Registration System designed to replace a legacy paper-based system. The documentation covers 14 Unified Modeling Language (UML) diagrams spanning requirements modeling, structural design, architectural/deployment mapping, and behavioral specifications, fully modeled within Visual Paradigm.

System Actors & Scope

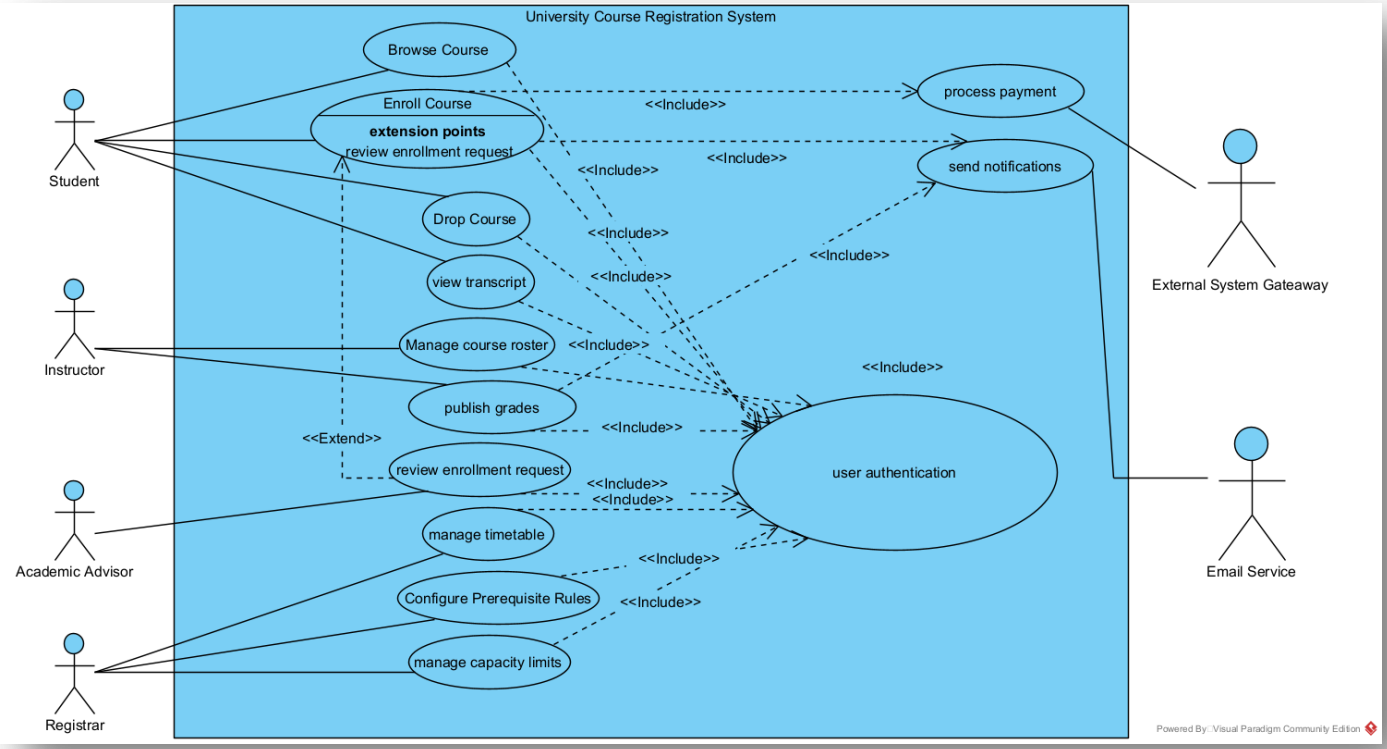
Briefly list down the 4 actors from your case study:

1. **Student:** Browses, enrolls in, and drops courses.
2. **Instructor:** Manages course rosters and publishes student grades.
3. **Academic Advisor:** Approves or blocks student enrolment requests.
4. **Registrar:** Exercises administrative control over timetables, capacity limits, and prerequisite rules.

Phase 1: Requirements Modeling

Use Case

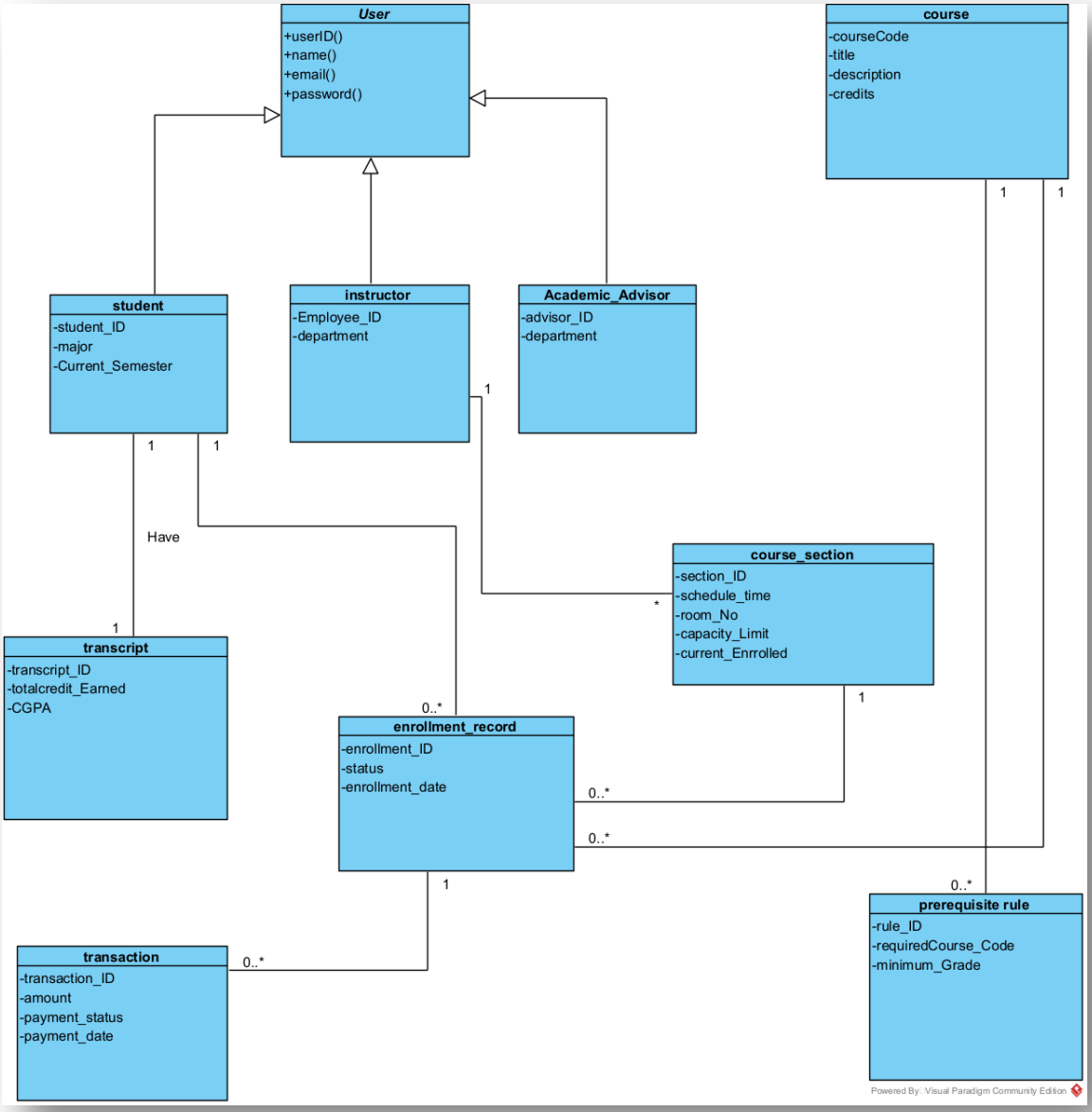
This diagram captures the system boundary and functional requirements of UniReg. It models 4 core actors interacting with 12+ use cases (e.g., *Enroll in Course*, *Publish Grades*, *and Manage Rosters*). It incorporates <<include>> relationships for mandatory authentication sequences and <<extend>> relationships for handling exceptional paths like *Payment Failure*.



Phase 2: Structural Design

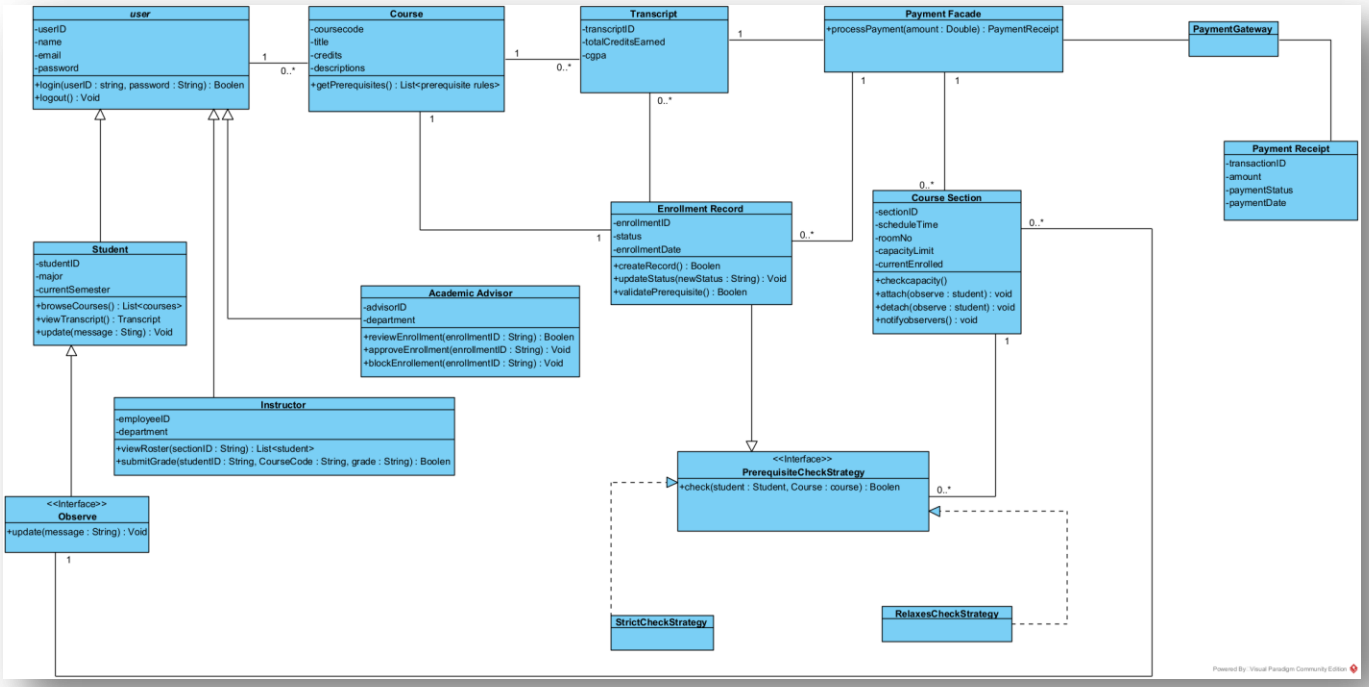
Domain Class Diagram

Represents the high-level conceptual model of the domain. It includes 10+ core entities such as Student, Instructor, Course, Section, Enrolment Record, and Registrar. It specifies structural relationships using associations, precise multiplicities (e.g., 1-to-many between Course and Section), inheritance structures, and aggregation (e.g., a Course containing multiple Prerequisite Rules).



Design Class Diagram

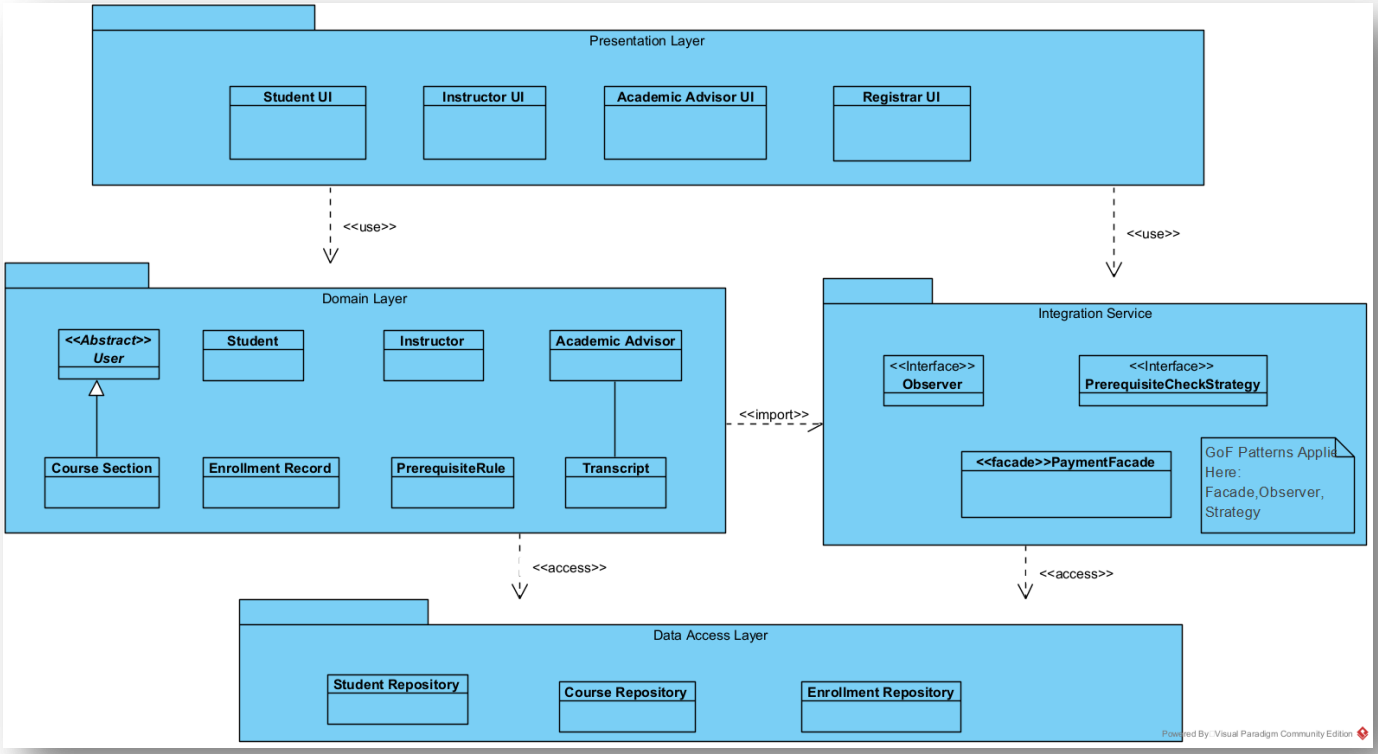
An evolution of the domain model containing implementation-level details. It explicitly defines method signatures (e.g., verify Prerequisites (), process Payment ()), visibility modifiers (+ for public, - for private), and interface abstractions. It clearly demonstrates the realization of GoF design patterns via stereotyped classes.



Phase 3: Architectural & Deployment Mapping

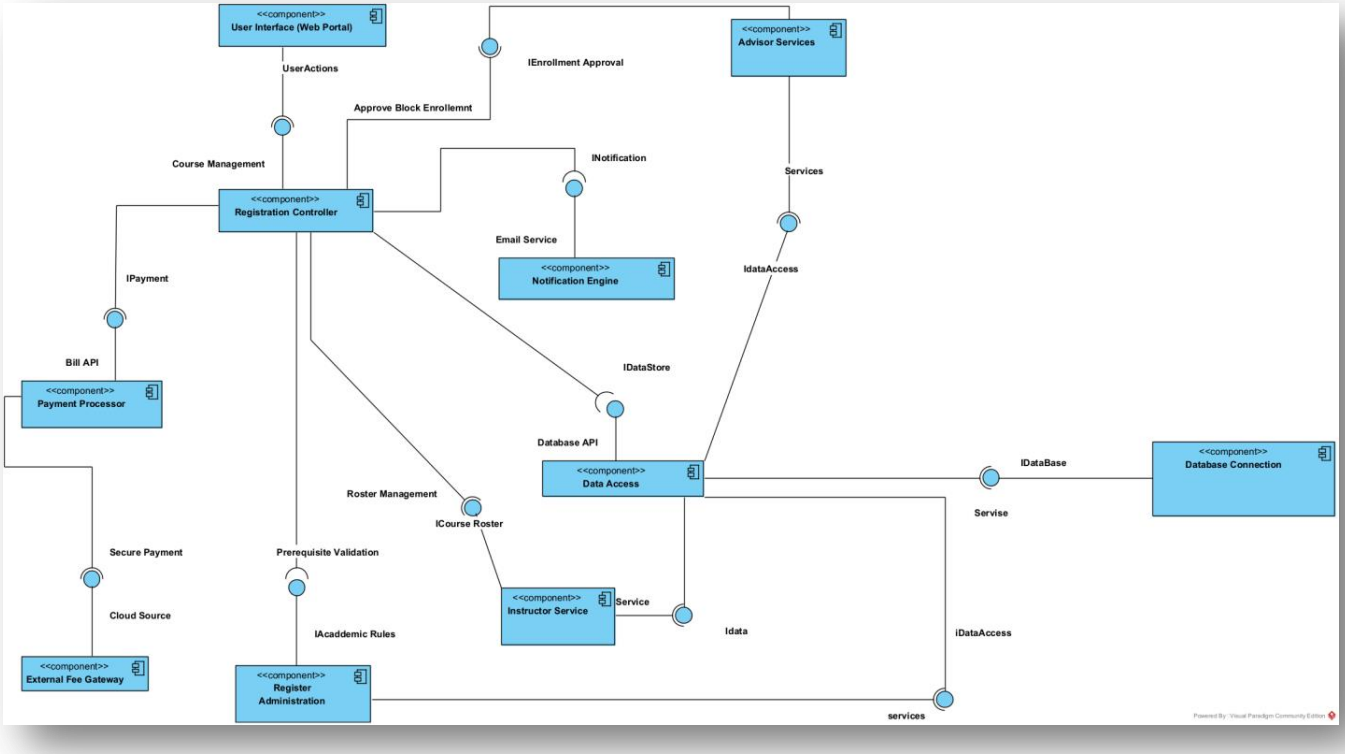
Package Diagram

Illustrates the architectural organization of system classes into 4 distinct subsystems/packages: UserManagement,EnrolmentEngine,AcademicRecordSubsystem, and Integration Gateway. Dependency arrows between packages are explicitly labeled with architectural stereotypes.



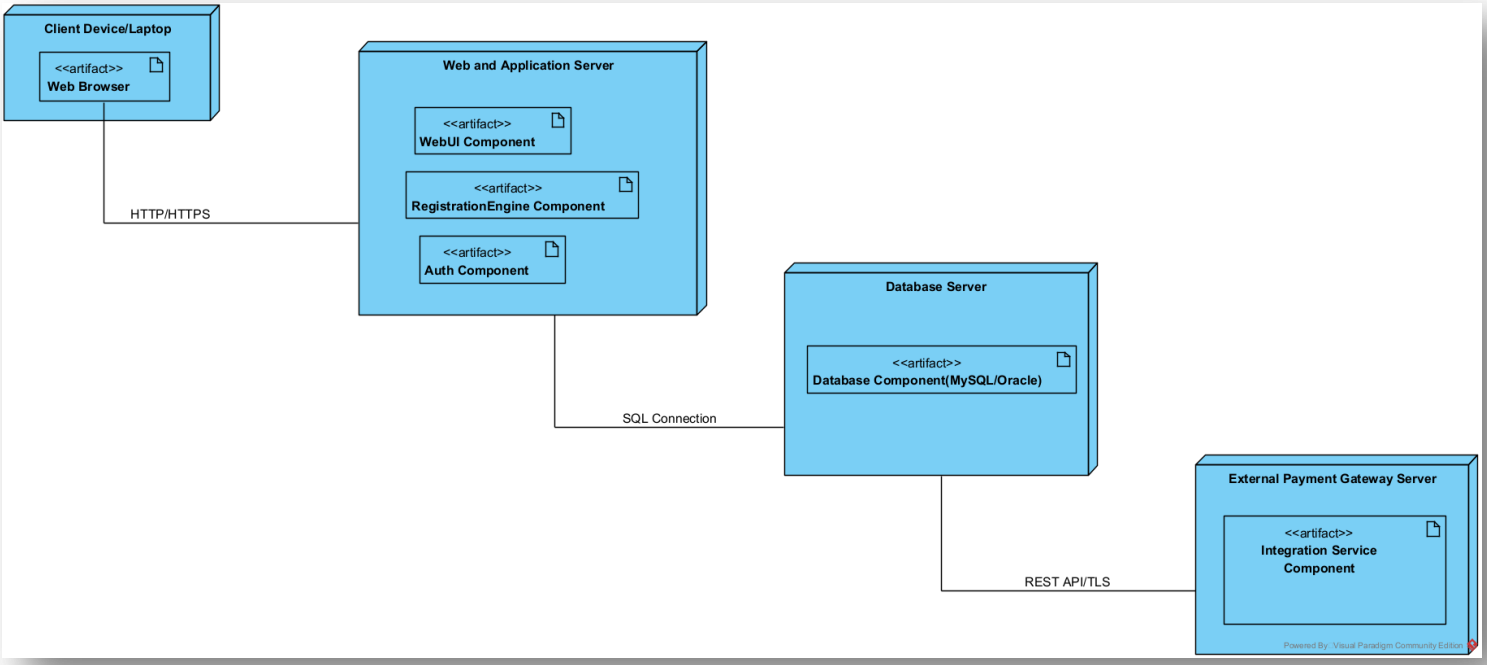
Component Diagram

Captures the component-based architecture of UniReg. It highlights components like Web UI, Registration Controller, NotificationService, and PaymentGatewayInterface interconnected via lollipop-and-socket configurations representing provided and required interfaces.



Deployment Diagram

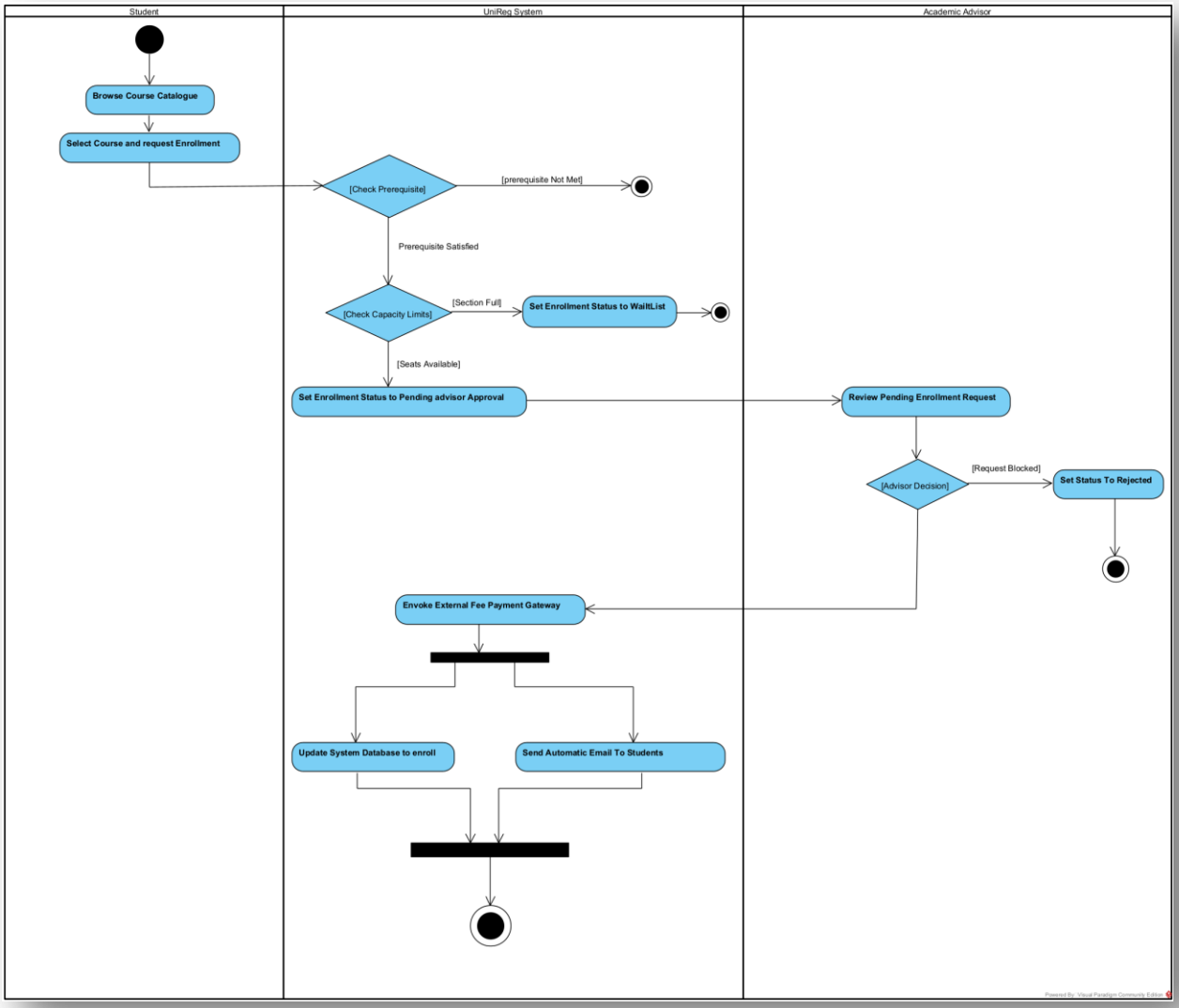
Maps the physical execution environment of the platform. It features physical infrastructure nodes (Client Web Browser, Application Server, Database Server, and External Payment Cloud API) and manifests executable software artifacts onto their respective hosting environments.



Phase 4: Behavioral Specifications

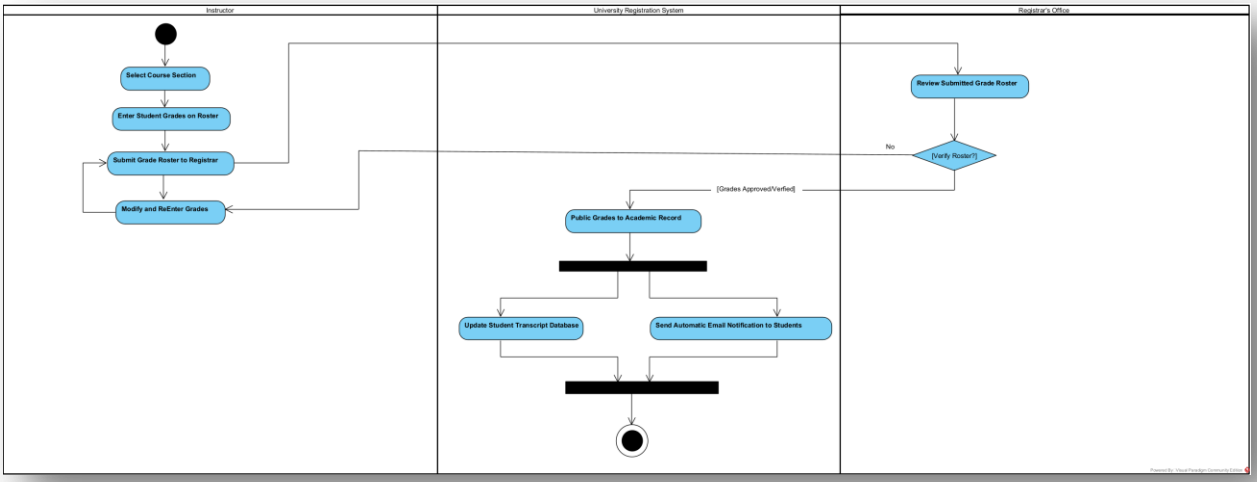
Activity Diagram Enrollment

Outlines the workflow logic for course enrollment mapped across distinct swim-lanes (Student, Enrolment Controller, and DB). It incorporates decision nodes to handle automated runtime checks for course capacity and prerequisite validation.



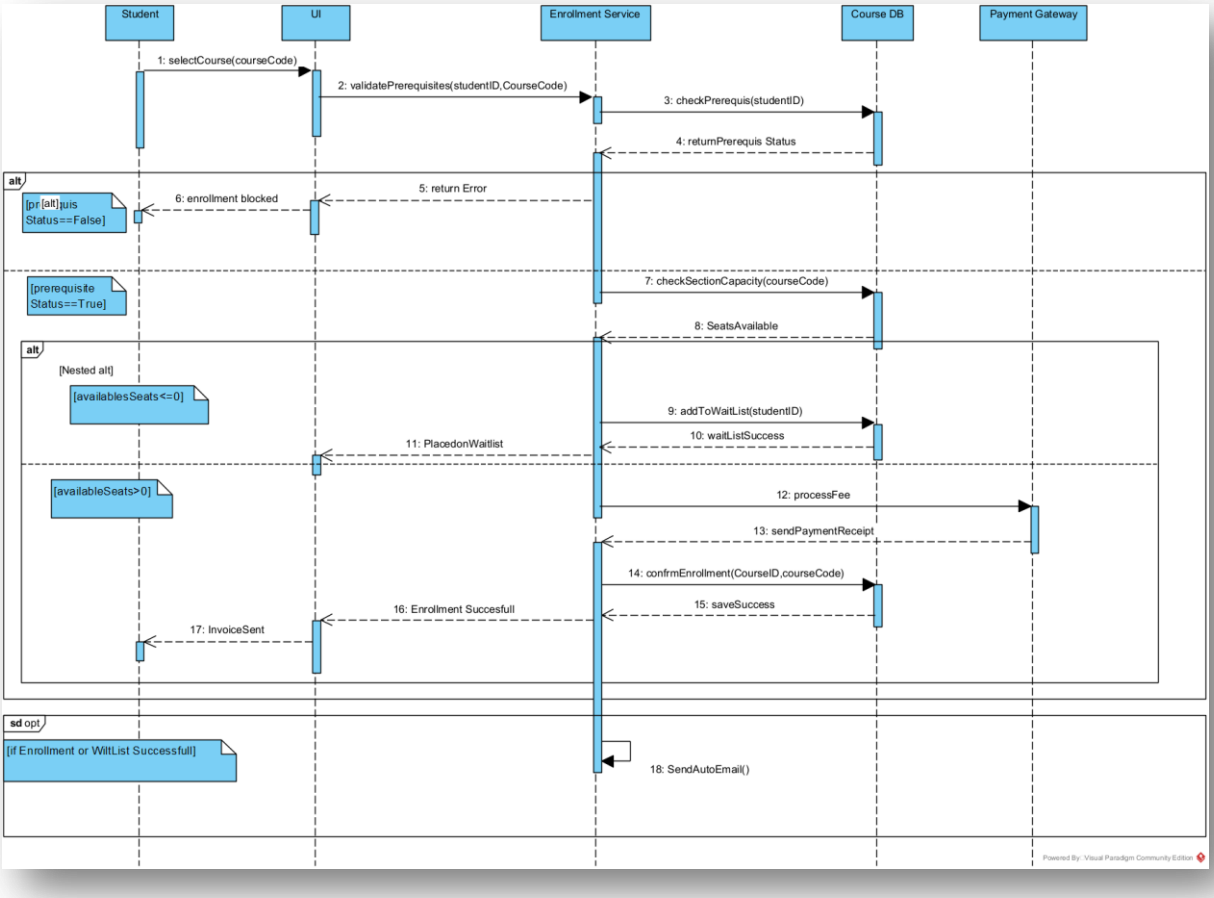
Activity Diagram Grade Entry

Details the workflow of grade submissions by an Instructor, followed by administrative auditing and approval by the Registrar. It utilizes fork and join synchronization bars to trigger concurrent notifications across email and student portal channels.



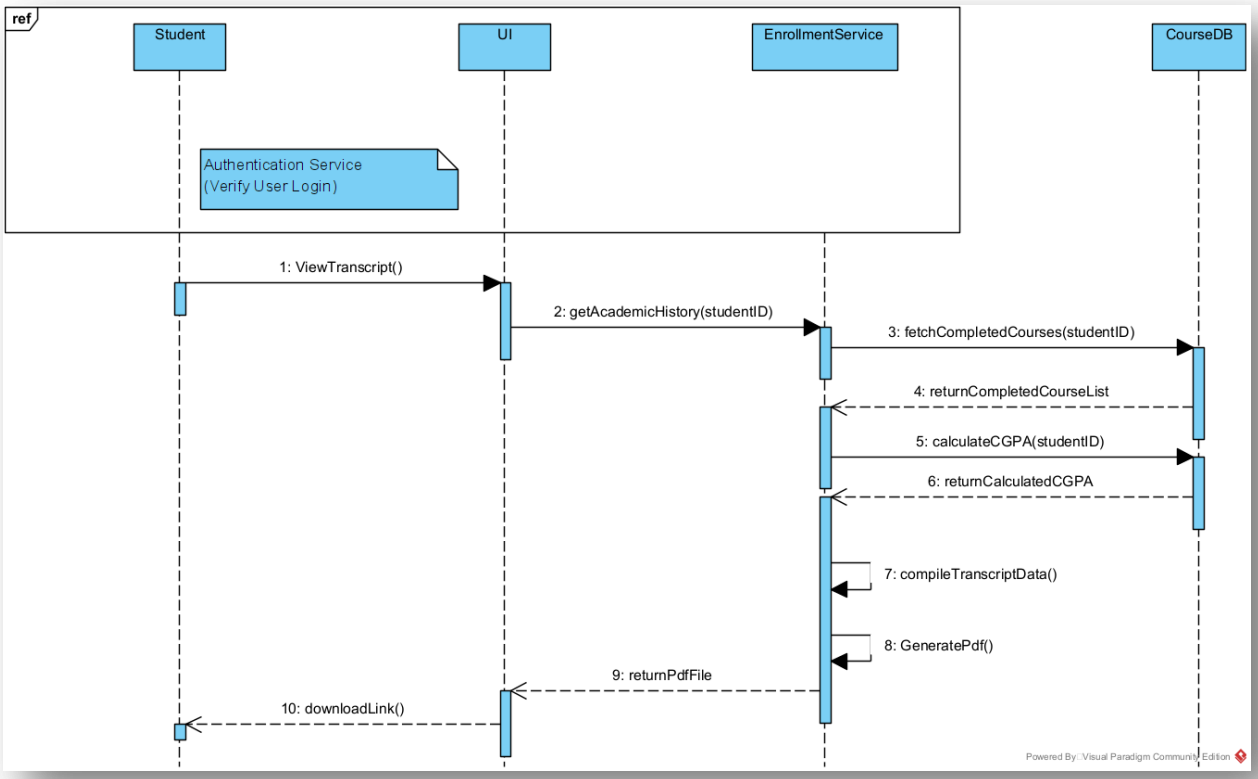
Sequence Diagram Enrollment

Chronologically charts object interactions during an active enrolment. It features life-lines for the Student actor, WebUI boundary, EnrolmentService controller, CourseDB entity, and PaymentGateway proxy, utilizing alt and opt conditional execution fragments to handle success and roll-back workflows.



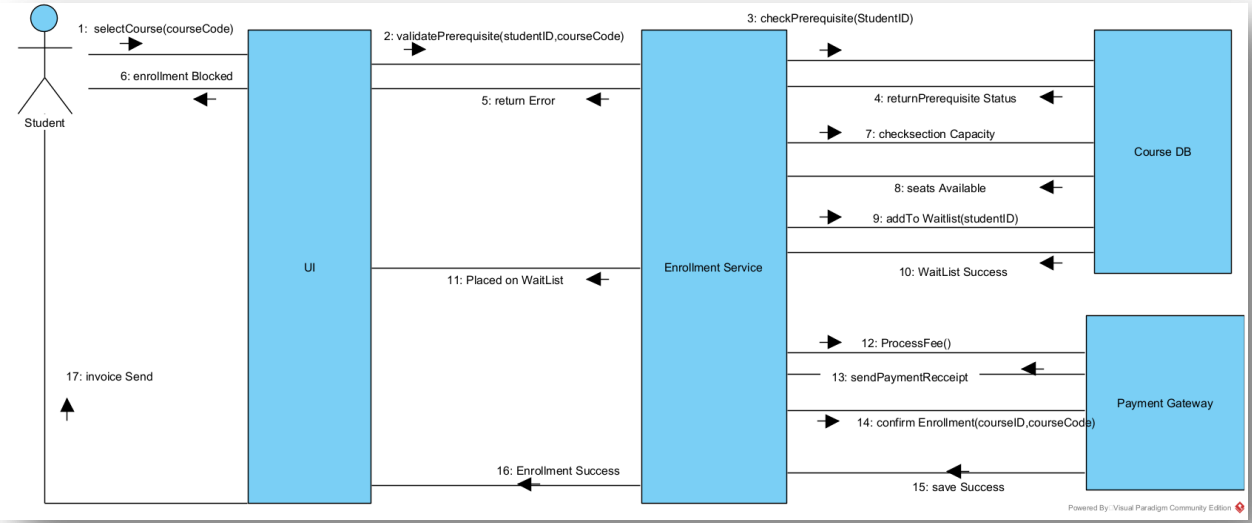
Sequence Diagram Transcript Generation

Details the data assembly flow when a student requests academic records. It illustrates the interaction from request handling to background database queries, culminating in a dedicated self-callback message for PDF generation, with a ref fragment linking back to the base authentication sequence.



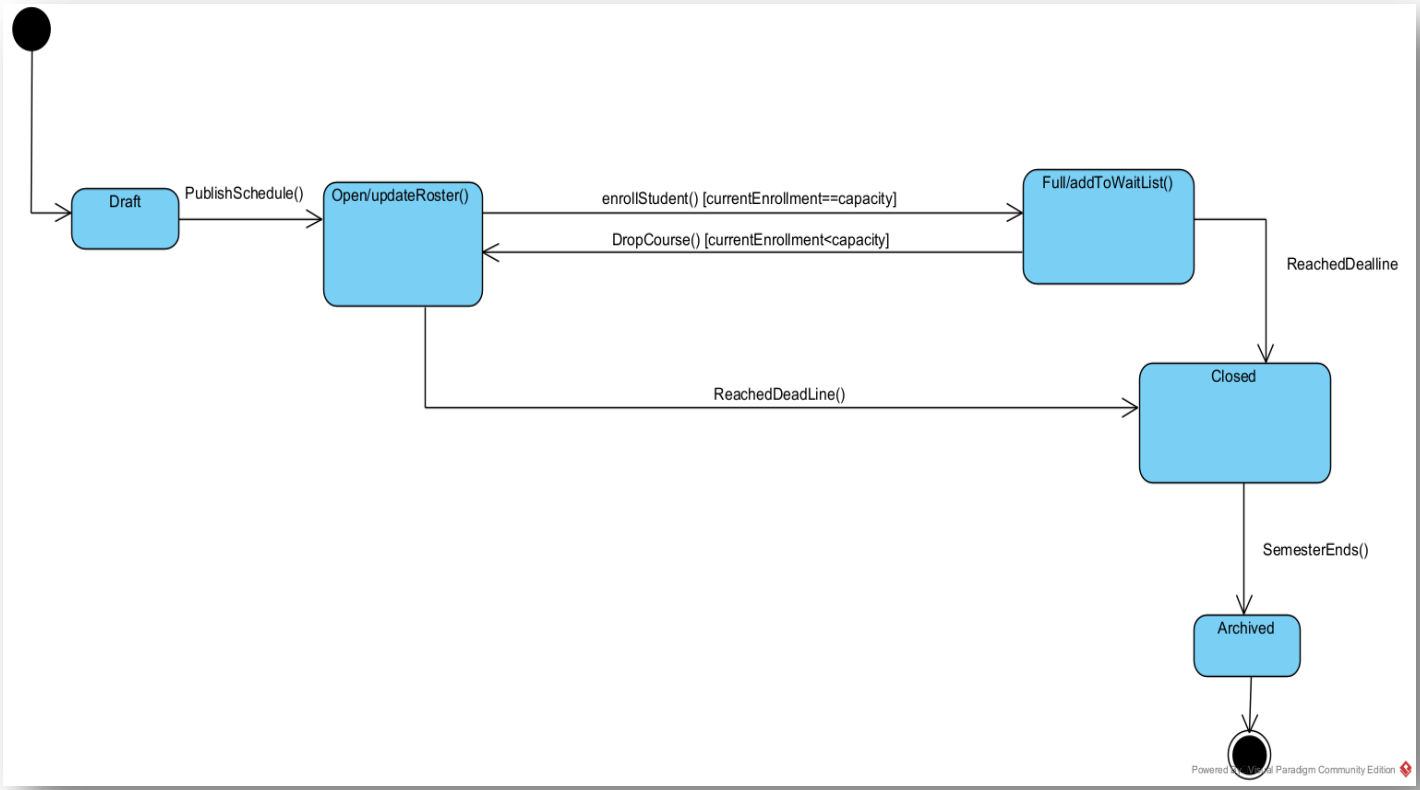
Communication Diagram

An alternative semantic representation of the *Enroll in Course* workflow. It emphasizes object link organization and explicitly numbers sequential message tokens to illustrate structural message passing.



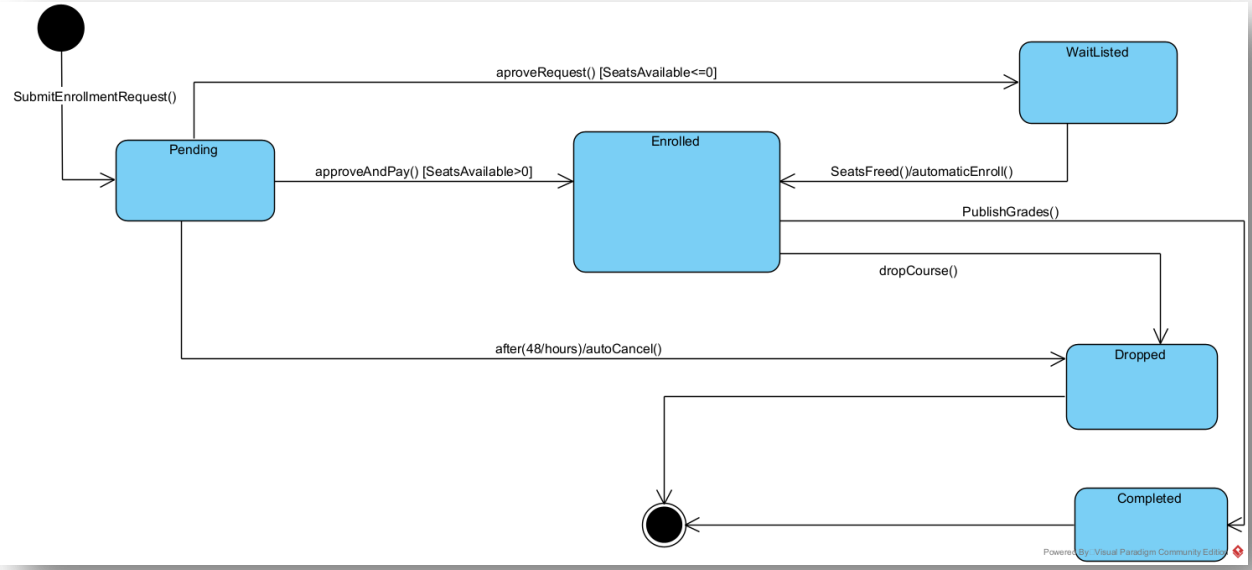
State Machine Course Status

Models the lifecycle states of a course instance (Draft ➔ Open ➔ Full ➔ Closed ➔ Archived), driven by behavioral events, internal actions, and strict guard conditions.



State Machine Enrollment Record

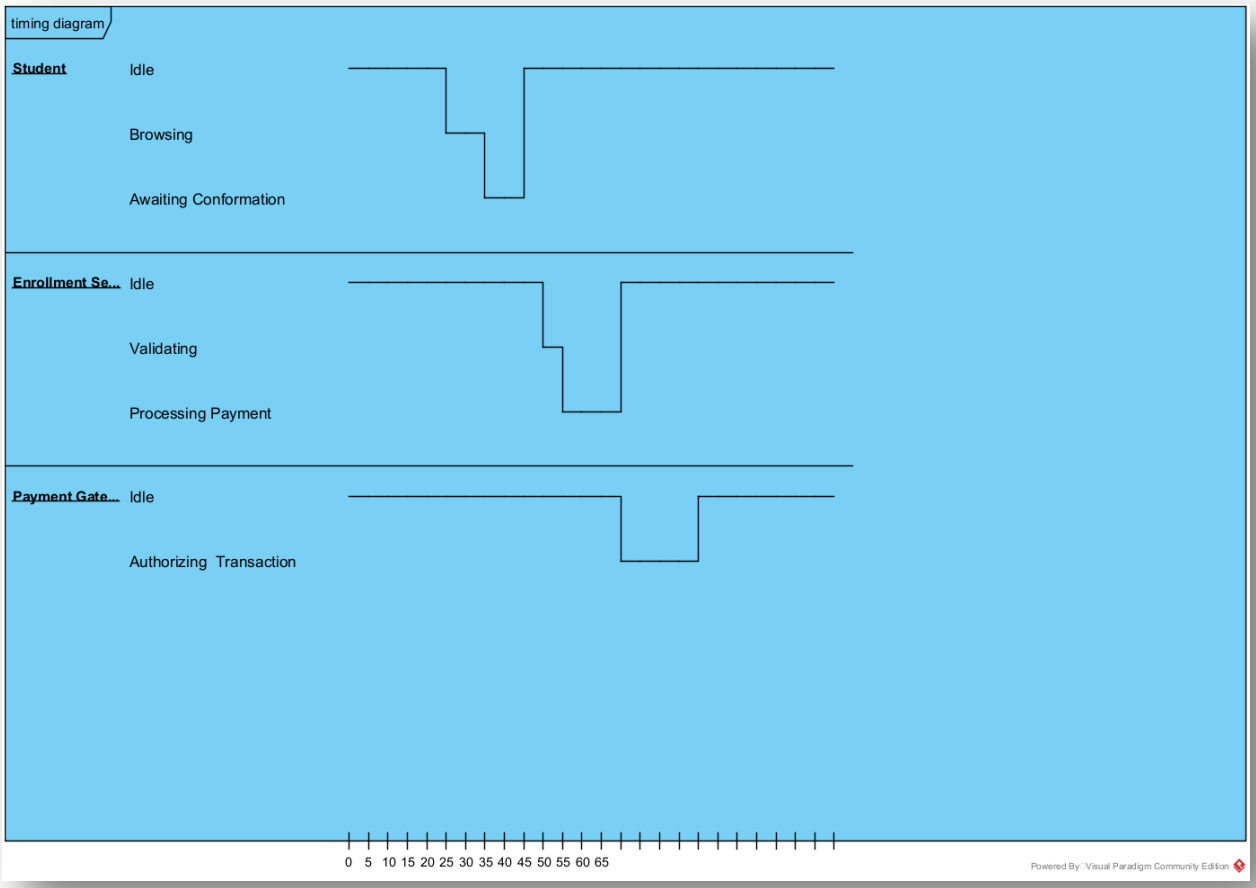
Defines the discrete transactional states of an individual enrolment entry (`Pending` ➔ `Enrolled` ➔ `Waitlisted` ➔ `Dropped` ➔ `Completed`), including transition mechanisms triggered by system timeouts.



Timing Diagram

A strict timeline analysis tracking concurrent state changes over a linear time axis for three main lifelines: Student, Enrolment Service, and Payment Gateway.

It precisely highlights object execution states such as Idle, Browsing, Validating, and Processing Payment. The timeline models the exact latency and state transitions when a student triggers an enrollment, showing how the service state shifts dynamically into active validation and waits for the external gateway to authorize the transaction before returning to an idle state synchronously.



Mandatory Design Patterns

Observer Pattern

Applied to decouple the core system from notification delivery mechanisms. When a Course changes status to Open, or an EnrolmentRecord transitions to Completed (grade posted), it automatically fires updates to all subscribed Student objects without tightly coupling the domain logic to the email server.

Strategy Pattern

Applied within the `EnrolmentEngine` to handle dynamic validation. Prerequisite checking rules can vary drastically based on

department policies. By encapsulating these rules inside interchangeable strategy classes (e.g., `StandardPrereqStrategy`, `AdvancedStandingStrategy`), the registrar can alter runtime rules dynamically without mutating core application code.

Facade Pattern

Implemented to manage integration with the external fee payment gateway. Instead of exposing complex, shifting multi-step API structures of third-party systems throughout the codebase, a unified `PaymentGatewayFacade` is introduced, providing a clean, single-method interface (`processTransaction()`) to the `EnrolmentService`.